



Aplicação de Machine Learning para detecção de ameaças em dispositivos de Internet das Coisas

Alessandra Carolina Domiciano

Ana Paula Serafim de Góis

A glowing green padlock is positioned on the left side of the slide, set against a dark background with a complex, glowing circuit board pattern. The padlock itself is composed of many small, bright green dots, giving it a digital or pixelated appearance. The circuit lines are thin and light blue, crisscrossing the dark background.

Introdução

- A expansão da Internet das Coisas (IoT) traz consigo desafios relacionados a segurança cibernética.
- A Análise de Tráfego de Rede baseada em Machine Learning (ML) é uma área de estudos promissora para segurança de IoT.
- Modelos de ML oferecem maior flexibilidade na identificação de ataques previamente desconhecidos e podem ser otimizados para operar com menor sobrecarga computacional.



Introdução

- Alguns desafios do uso de ML para detecção de ameaças de IoT:
 - Criação de bases de dados para treinamento
 - Desbalanceamento de classes
 - Alta dimensionalidade dos dados
 - Dados massivos
 - Recursos computacionais limitados
 - Retreinamentos
- Os modelos de ML devem ser leves e rápidos, com boa capacidade de generalização e desempenho equilibrado, com um bom compromisso entre sensibilidade e especificidade.

Introdução

- Este trabalho se baseia no modelo proposto em [3], e **seu objetivo é construir um ambiente de teste para avaliação da proposta.**
- Para tanto, foi implementada uma rede IoT com dispositivos físicos e o sistema de teste do modelo foi executado em uma Raspberry Pi 4.
- Observaram-se os resultados das métricas de classificação binária, priorizando F1 score e latência de predição.



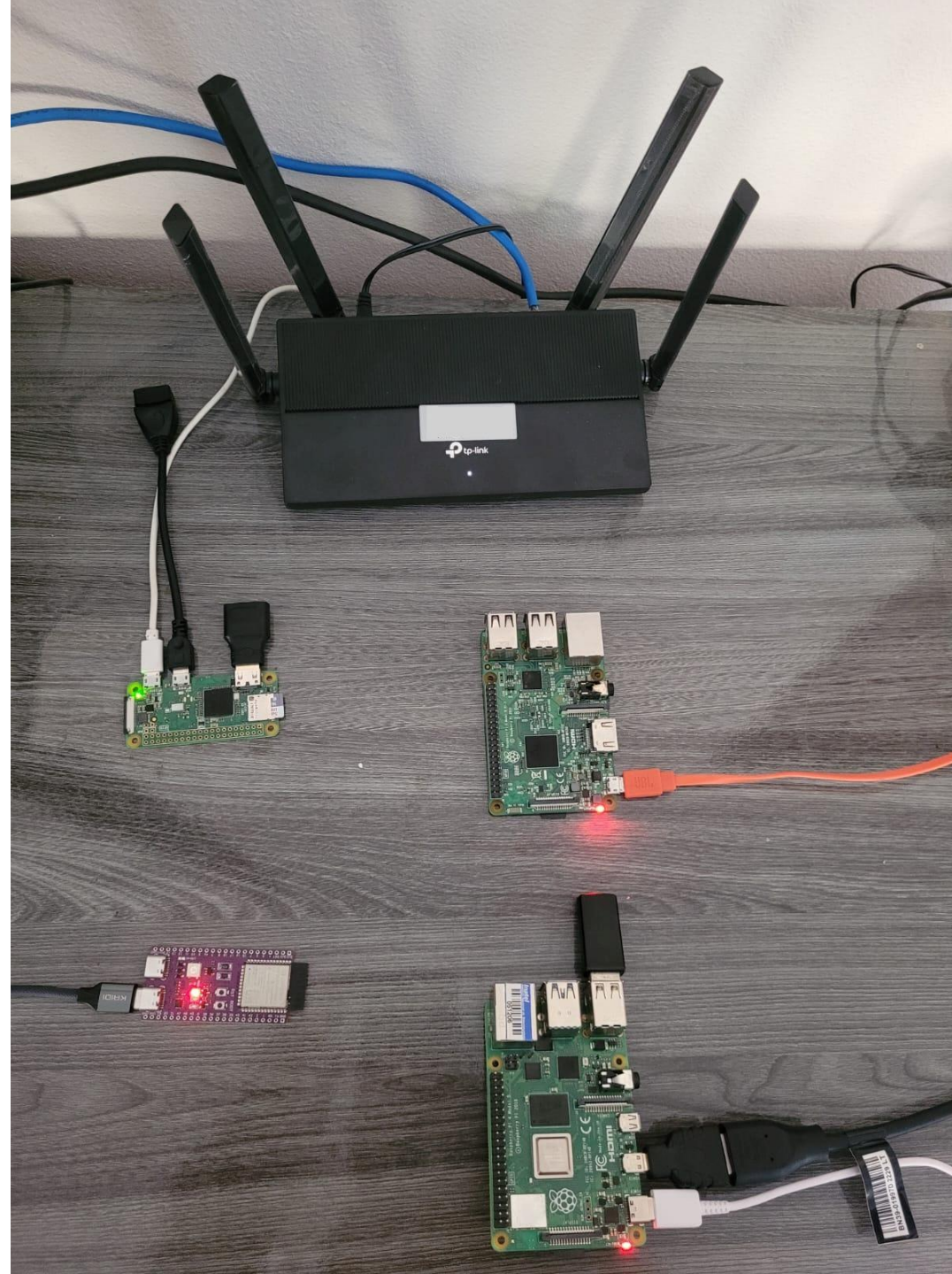
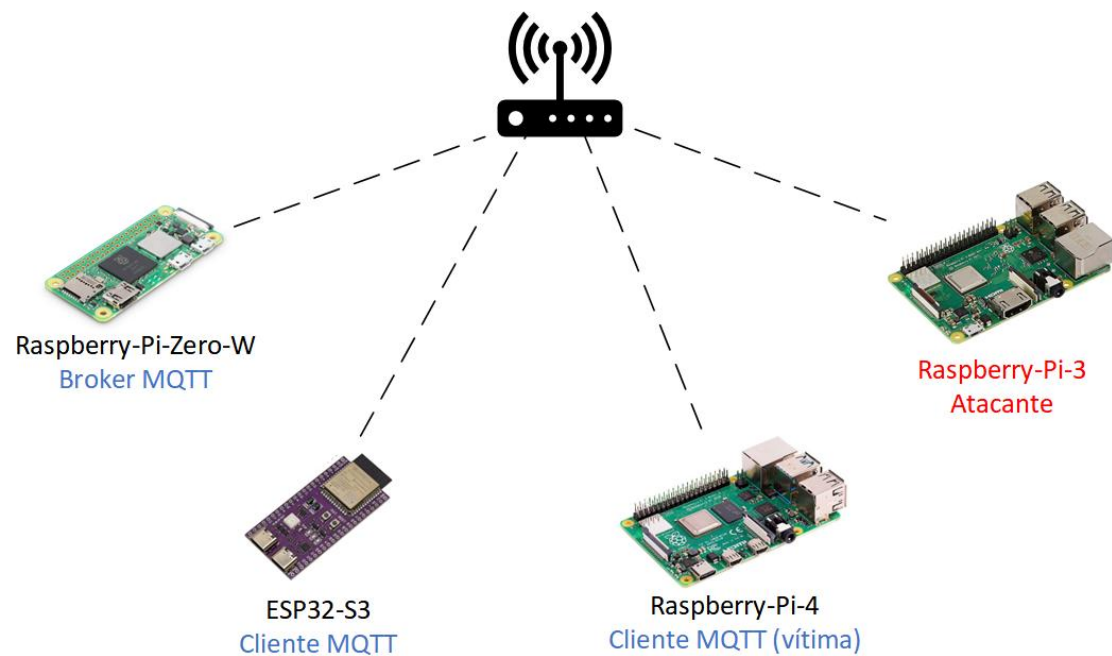
Metodologia

O desenvolvimento deste trabalho pode ser dividido em três etapas:

1. Provisionamento da rede IoT.
2. Desenvolvimento do sistema de teste.
3. Retreinamento do modelo.

Metodologia

Provisionamento da rede IoT:



Metodologia

Provisionamento da rede IoT:

- **Raspberry Pi Zero W:** executa um broker MQTT.
- **ESP32:** executa um cliente MQTT que publica dados em um tópico de teste e assina esse mesmo tópico.
- **Raspberry Pi 4 (vítima):** executa um cliente MQTT que publica dados em um tópico de teste e assina esse mesmo tópico. **É quem executa o sistema de testes.**
- **Raspberry Pi 3 (atacante):** dispara os ataques contra a vítima.

Metodologia

Ferramentas:



Varredura agressiva (Scan A)

Portas TCP abertas, fechadas e filtradas

Versões de serviços

Sistema operacional

Scripts adicionais

Rota até o host



Varredura UDP (Scan sU)

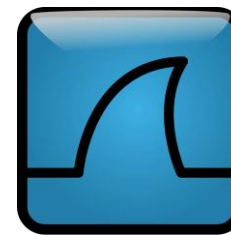
Portas UDP abertas, fechadas e filtradas



Força bruta SSH (Sparta)

Testa várias combinações de credenciais para se autenticar em um serviço SSH

Utilizada wordlist com 500 senhas frequentes

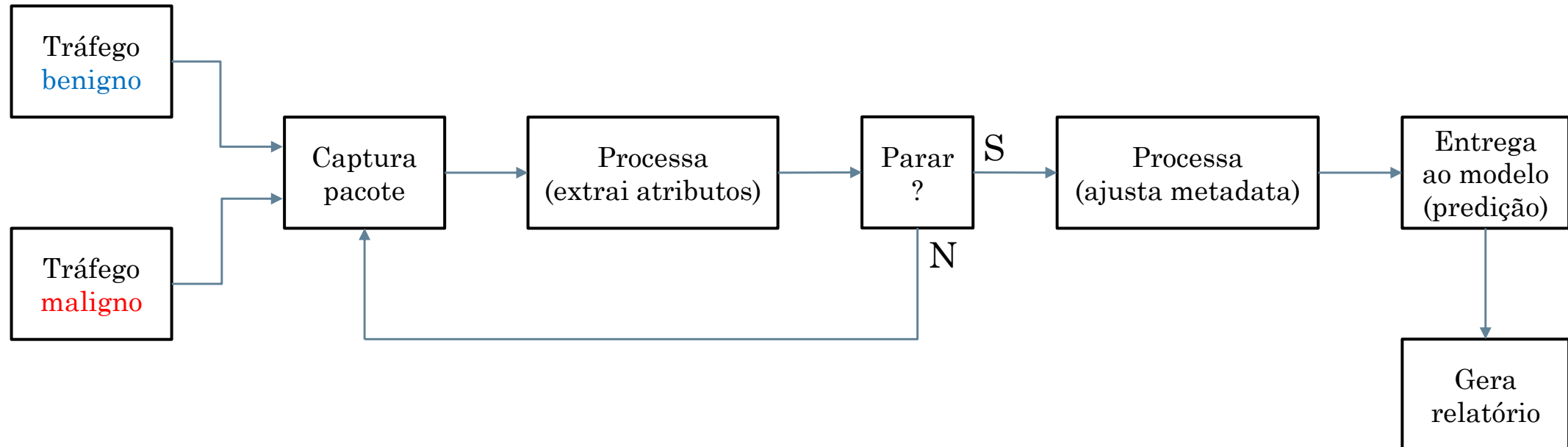


Tshark

Captura de tráfego de rede

Metodologia

Sistema de teste:



Metodologia

Sistema de teste:

1. O sistema inicia realizando a captura do tráfego de rede que entra e sai da interface do dispositivo vítima;
2. Prossegue fazendo o processamento dos dados, que se trata da extração dos atributos. São extraídos 30 atributos, que consistem em campos de cabeçalho dos protocolos IP, TCP ou UDP e MQTT.
3. Esse processo é realizado pacote a pacote até que o usuário pare a captura.
4. Uma vez que isso ocorre, o conjunto de dados é submetido a um novo processamento, desta vez, para ajustar os valores dos atributos e selecioná-los conforme o metadata utilizado no treinamento do modelo.
5. Por fim, os dados são entregues ao modelo, que é integrado ao sistema, para realizar as previsões e gerar as métricas de avaliação.

Metodologia

Retreinamento do modelo:

Algoritmo
XGBoost

Baseado em árvores de
decisão

Empregado tanto para
regressão quanto para
classificação

Combina diversos modelos
fracos para formar um
modelo forte (ensemble)

Pipeline HPO
e
pré-processamento

Amostragem do dataset em
10%

Seleção de atributos

Filtragem de amostras

HPO para melhor latência,
melhor F1-score e melhor
equilíbrio entre ambos

Dataset
MQTT-IoT-
IDS2020

Dados gerados a partir de uma
rede simulada:

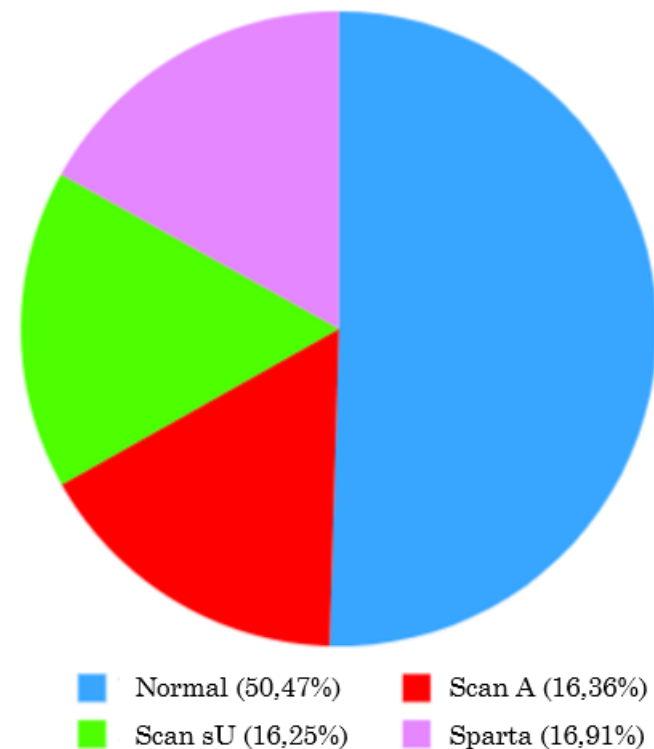
- Tráfego normal: 10,35%
- Scan A: 0,18%
- Scan sU: 0,1%
- Sparta: 89,36%

Metodologia

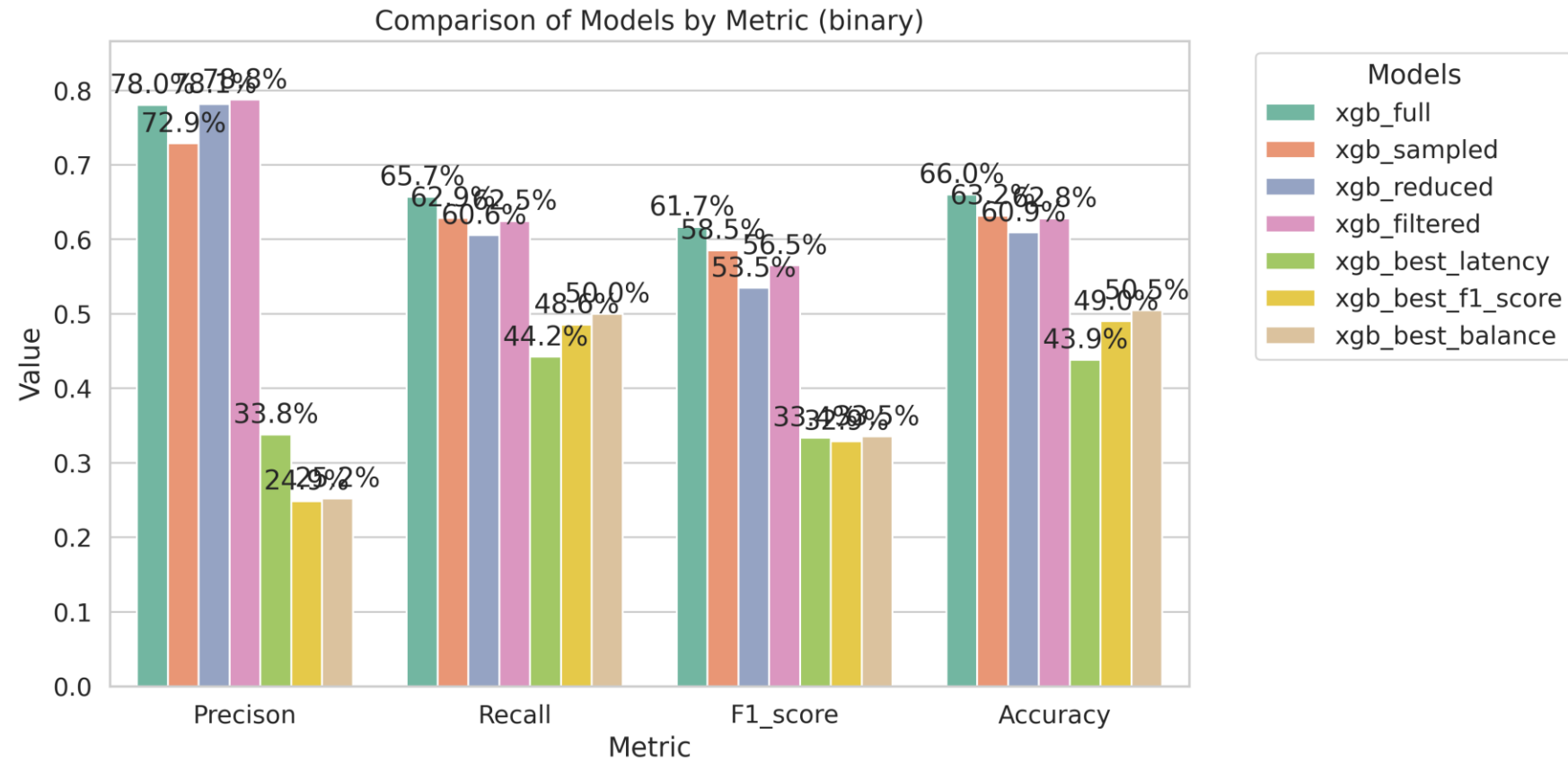
- **xgb_full:** é a linha de base, utiliza o dataset puro.
- **xgb_sampled:** realiza amostragem do dataset em 10% incluindo ao menos uma amostra de cada classe.
- **xgb_reduced:** realiza amostragem do dataset e seleção de atributos preservando os que atingem mais de 95% de ganho de informação.
- **xgb_filtered:** realiza amostragem do dataset, seleção de atributos e filtragem de amostras com base na média das frequências relativas dos valores dos atributos.
- **xgb_best_latency:** realiza amostragem do dataset, seleção de atributos, filtragem de amostras e otimização de hiperparâmetros com objetivo de minimizar a latência de predição.
- **xgb_best_f1_score:** realiza amostragem do dataset, seleção de atributos, filtragem de amostras e otimização de hiperparâmetros com objetivo de minimizar o F1 score.
- **xgb_best_balance:** realiza amostragem do dataset, seleção de atributos, filtragem de amostras e otimização de hiperparâmetros com objetivo de equilibrar a latência de predição e o F1 score.

Resultados e discussão

- O experimento consistiu em enviar tráfego de rede benigno e maligno ao dispositivo vítima, a fim de verificar o desempenho do modelo em um cenário real e diferente do que gerou os dados de treinamento.
- Ao todo, o experimento durou em torno de 1h e foram gerados 26520 pacotes, com a distribuição ao lado.
- Buscou-se produzir um equilíbrio entre as classes e entre os ataques. Para tanto, os ataques Sparta, Scan A e Scan sU foram executados uma, duas e duas vezes, respectivamente, enquanto o tráfego MQTT fluía continuamente.



Resultados e discussão



Resultados e discussão

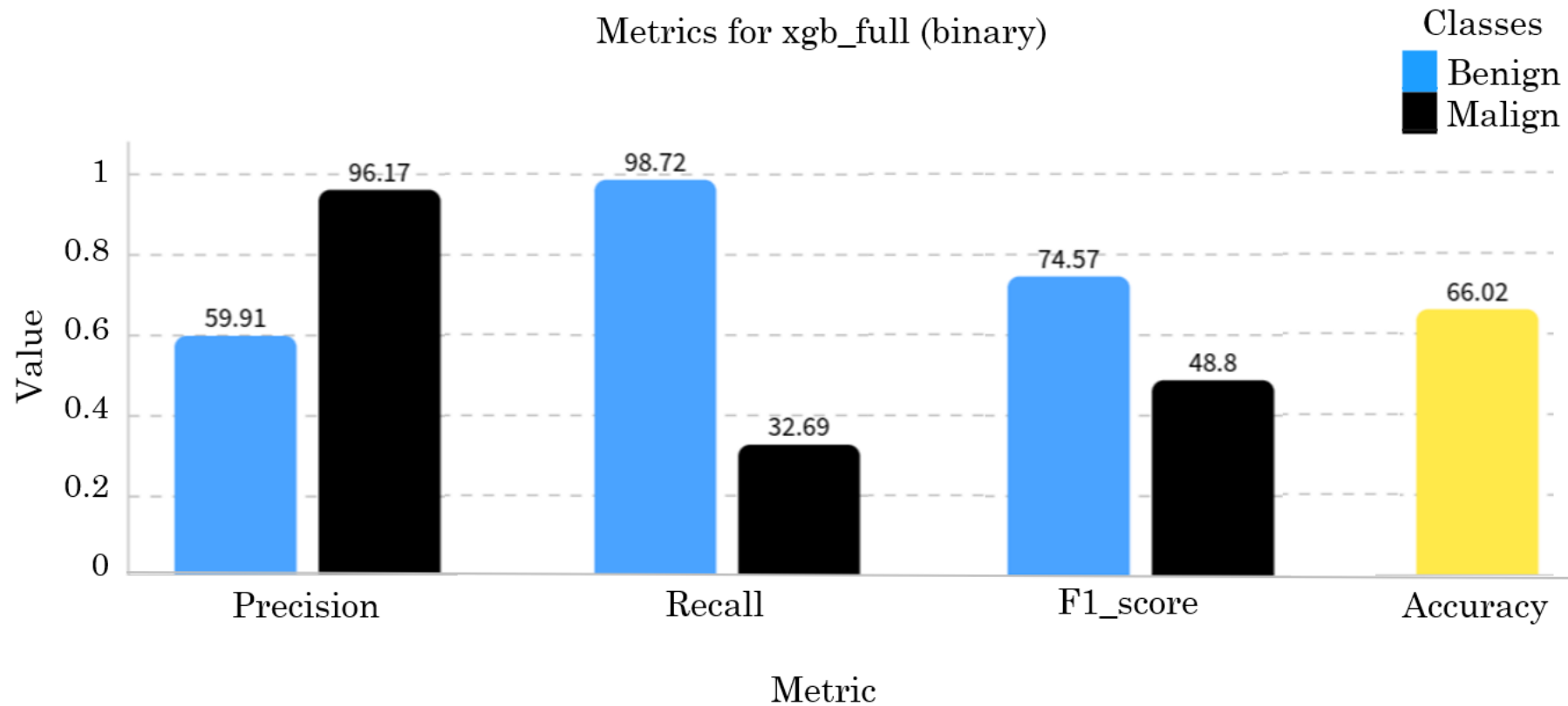
De maneira geral, os quatro modelos com melhor desempenho apresentaram resultados semelhantes, com pequenas variações.

	Precisão	Recall	F1 score	Acurácia
xgb_full	78,0%	65,7%	61,7%	66,0%
xgb_sampled	72,9%,	62,9%	58,5%	63,2%
xgb_reduced	78,1%	60,6%	53,5%	60,9%
xgb_filtred	78,8%	62,5%	56,5%	62,8%

Esses valores representam métricas médias e, portanto, não refletem as diferenças de desempenho entre as classes.

Optou-se por analisar mais detalhadamente o desempenho do modelo **xgb_full**, pois ele alcançou o maior F1-score.

Resultados e discussão



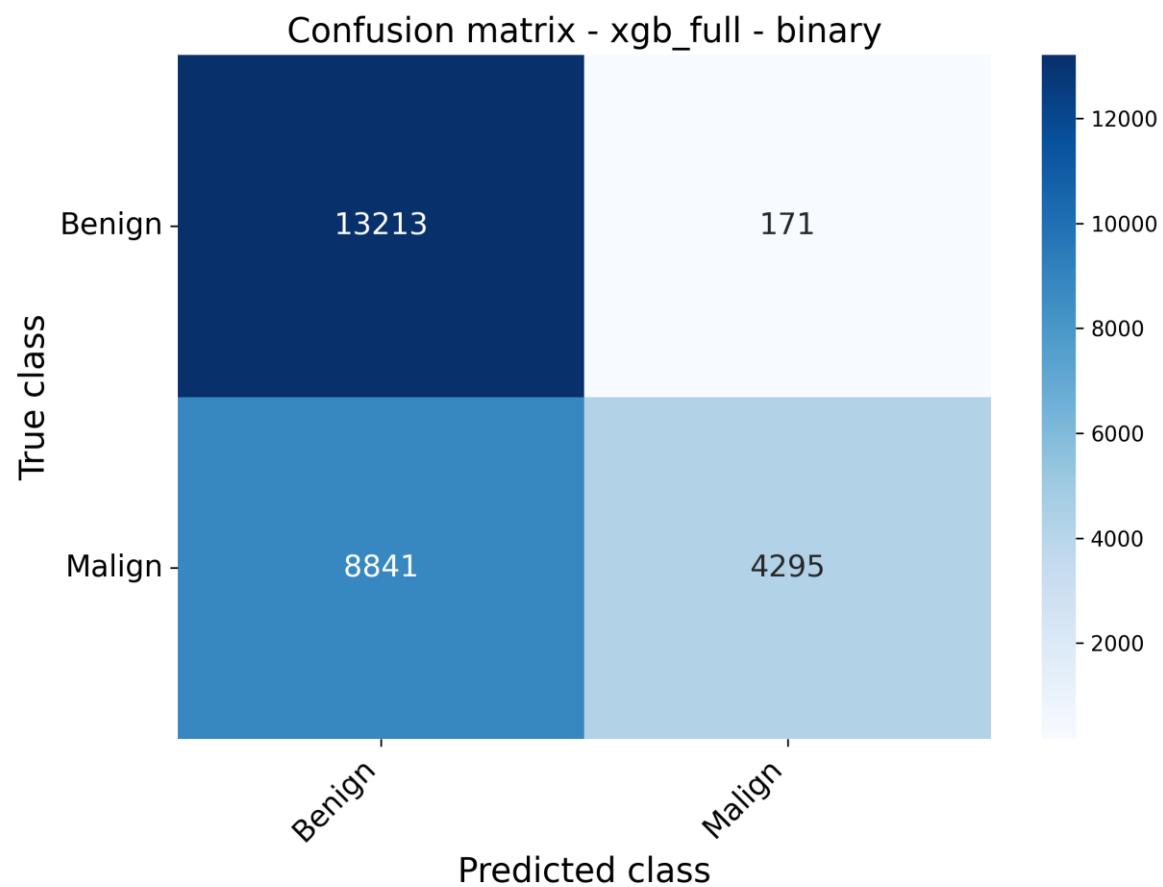
Resultados e discussão

- A análise dos resultados revela um **desempenho significativamente desequilibrado entre as classes**, com favorecimento da **classe benigna**.
- Isso é evidenciado pelo elevado recall (98,7%), indicando que quase todas as amostras benignas foram corretamente classificadas.
- No entanto, a precisão associada a essa classe é relativamente baixa (59,9%), sugerindo que o modelo produz um número considerável de falsos positivos.
- O desequilíbrio entre precisão e recall resulta em um F1-score de 74,6% para a classe benigna, o que, embora possa ser considerado razoável, mascara a fragilidade do modelo em relação à outra classe.

Resultados e discussão

- Já o desempenho para a **classe maligna** é bastante insatisfatório. Embora a precisão seja elevada (96,2%), o recall é extremamente baixo (32,7%).
- Isso indica que o modelo erra ao classificar mais de dois terços das amostras de ataques, o que implica uma alta taxa de falsos negativos.
- O F1-score para essa classe é de 48,8%, refletindo um desempenho baixo.
- As taxas de acertos e erros para cada classe podem ser melhor compreendidas observando a matriz de confusão.

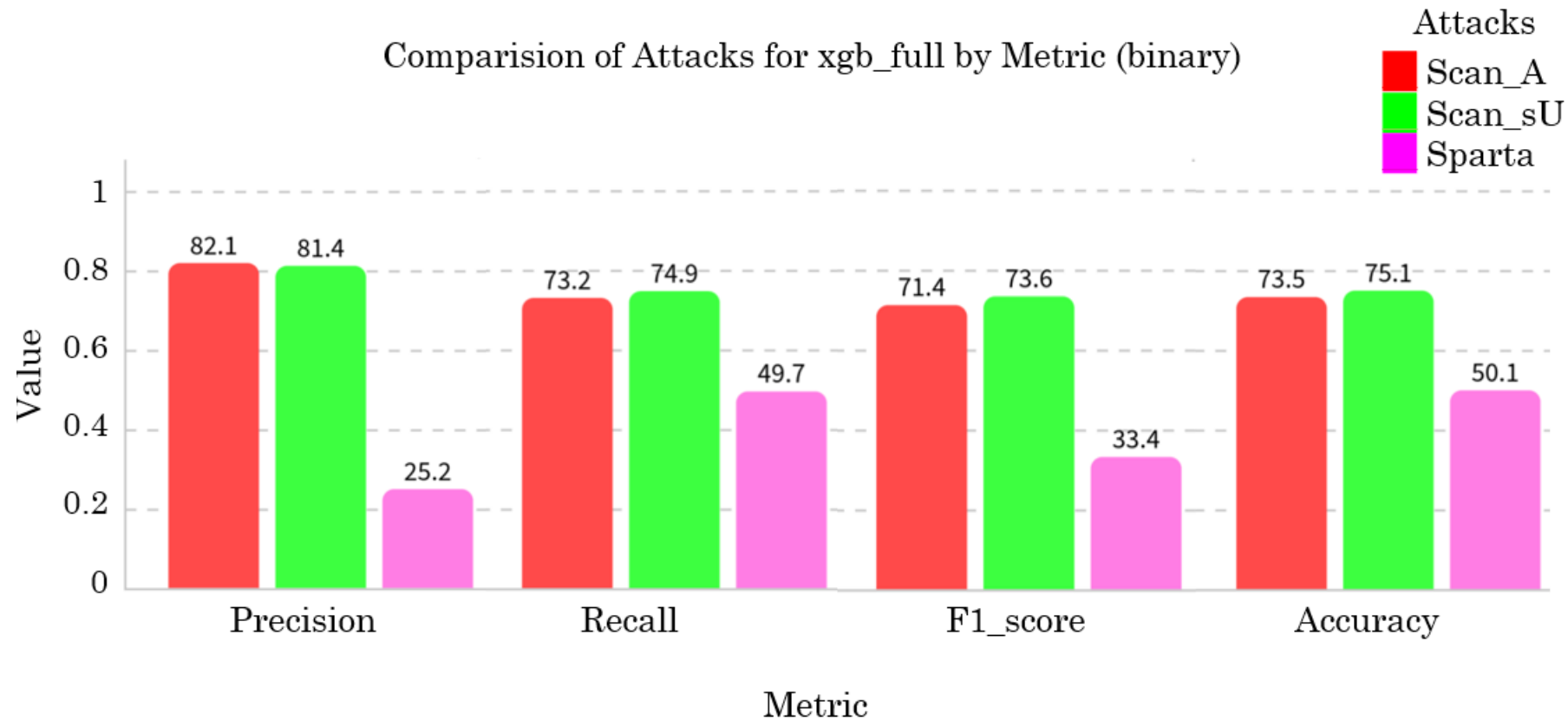
Resultados e discussão



Resultados e discussão

- Para uma análise ainda mais aprofundada, os dados de classificação gerados pelo modelo `xgb_full` foram armazenados em arquivo para permitir uma análise posterior do desempenho por tipo de ataque.
- Para isso, os dados foram segmentados em três subconjuntos, cada um contendo amostras de um único ataque específico, juntamente com tráfego benigno.
- A proporção entre classes em cada subconjunto foi mantida equilibrada, com cerca de 50% de amostras benignas.

Resultados e discussão



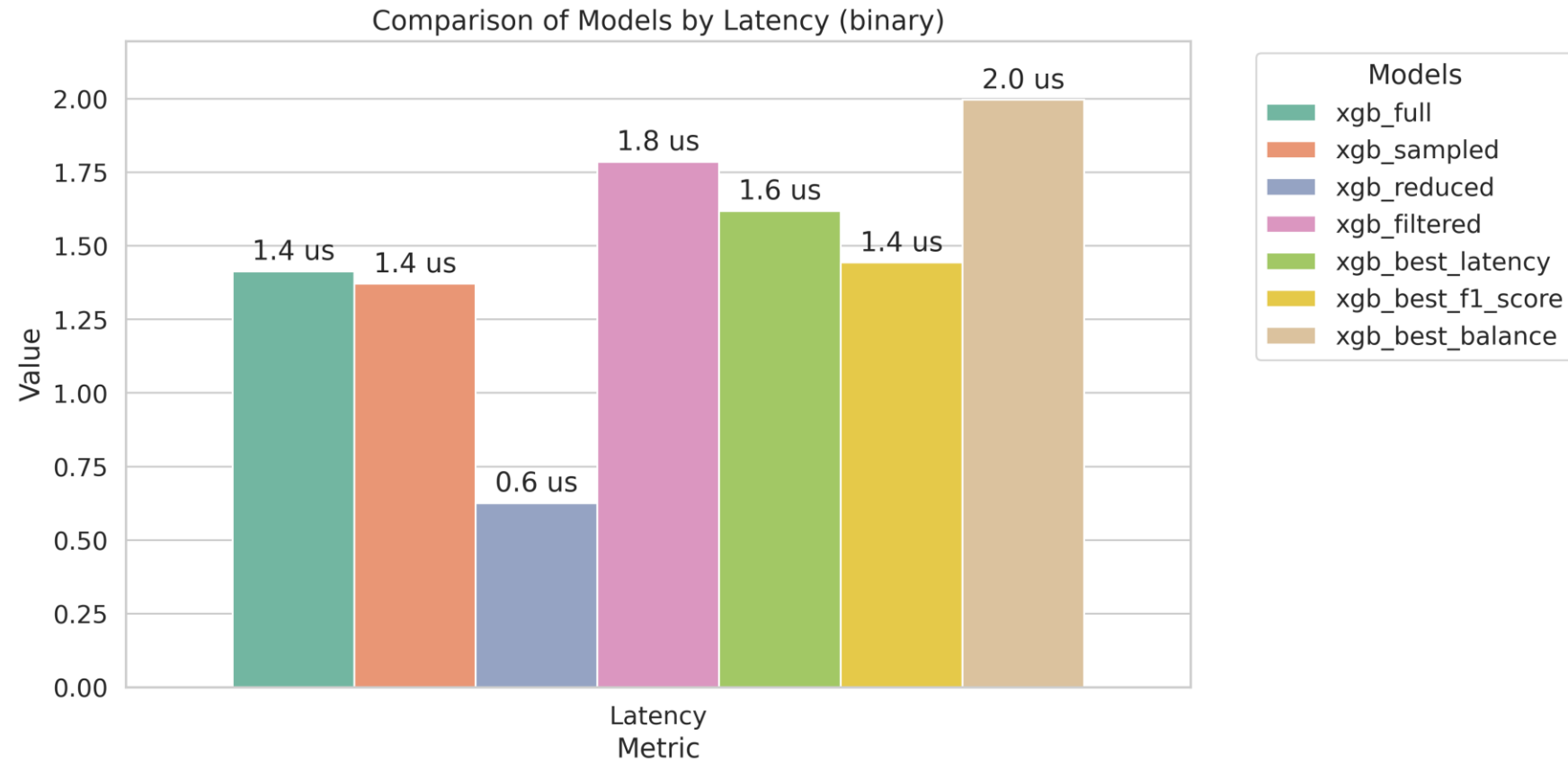
Resultados e discussão

Observa-se que o modelo apresenta um desempenho razoável nos testes com os ataques isolados Scan A e Scan U e um desempenho significativamente inferior para o ataque isolado Sparta.

	Precisão	Recall	F1 score	Acurácia
Scan A	82,1%	73,2%	71,4%	73,5%
Scan U	81,4%,	74,9%	73,6%	75,1%
Sparta	25,2%	49,7%	33,4%	50,1%

É importante salientar que tais métricas representam valores médios agregados, os quais podem ocultar desequilíbrios significativos no desempenho entre as classes benigna e maligna, conforme discutido na análise geral do modelo.

Resultados e discussão



Conclusão

- Foi criado um cenário realista e funcional para a execução de testes de detecção de ameaças cibernéticas em ambientes IoT, demonstrando a viabilidade de aplicação local de modelos de machine learning para tal finalidade.
- No entanto, os resultados evidenciam que o sistema desenvolvido ainda demanda melhorias significativas, sobretudo em relação à sua capacidade de identificar com precisão os ataques, devido a alta taxa de falsos positivos.
- Como trabalho futuro, propõe-se a integração dos processos de captura e pré-processamento de pacotes ao fluxo de predição, com o intuito de incluir essas etapas ao cálculo de latência média por amostra, o uso de outros bancos de dados e ameaças de IoT e a utilização de outros tipos de modelos, como os de IA generativa.



Obrigada!

Alessandra Carolina Domiciano

Ana Paula Serafim de Góis